计算机组成原理 · 指令寻址与汇编 · 考点精讲与大题整合

408 各科大题知识点强化课 | 课程第 7 讲（分 P9） | 时长 3 小时 26 分 45 秒

本讲范围：两条线 —— ①**指令系统（计组第四章 4.1–4.2）**：第一章回顾（MAR/MDR、三总线、编址方式、三个「字长」、时钟与 CPI）→ 指令格式（操作码 + 地址码、0～4 地址指令）→ **指令寻址**（顺序 / 跳跃，都是改 PC）→ **数据寻址十种**（隐含、立即、直接、间接、寄存器、寄存器间接、相对、基址、变址、堆栈）与 EA 的写法 → **基址 vs 变址**（谁变谁不变、程序浮动、数组）；②**汇编与机器级表示（4.3 起）**：185 页那张指令表 → **四道「C 代码 + 行号 + 虚拟地址 + 机器指令 + 汇编」大题**（本讲精讲两道，另两道他说课下录）。

读法建议：**§3 方法论**（判编址方式、三个字长怎么选、相对寻址真题算式 PC+2+2×偏移量、变址取数组、CF/OF 两条路）→ **§4**（他课上手推的六道题）→ **§7 速查卡**。§2 是概念主体（**寻址方式那一张表**最值钱），§5 是年份认定，§6 是易错点，**§附**里放了「算式独立验算清单」和「转写含糊清单」。

一句话：「**第四（章）典型的就是知识点很少**」「书上内容不多，**但是要通过题目去练习**」；「**组（原）的考法是最灵活的，尤其是指令和 CPU 这里**」——本讲值钱的是**书上那套「PC+A / BR+A / IX+A」在真题里全部要改口径**。

目录

1 考情分析：第四章的分量与做题观

2 核心概念（含第一章回顾）

3 解题方法论：五套判断与算式

4 典型题型与例题（他手推的六道）

5 真题年份认定与命题特点

6 易错点清单

7 考场速查卡

附 来源、边界与验算清单

ASR 校正说明（读引用前先看这一段）

转写稿由本地 Whisper 生成，术语同音错字密度极高，且本讲**全程无板书**（地址、位串、除法与补码都是口述的）。引用处已统一校正，主要替换：**主持人员里 / 主同原理 / 组成园里 / 左同原理 → 组成原理（计组）**；**大体 / 大铁 / 大币 / 提 / 帖 / 体 → 大题 / 题、整体 / 真体 → 真题**；**地质码 → 地址码**；**寻植 / 循指 / 寻指 / 选址 / 询子 / 巡止 / 勋子 → 寻址**；**基存器 / 计存器 / 技存器 / 基层级 / 基存级 / 纪存器 → 寄存器**；**机子 / 机质 → 基址、变质 / 变子 / 辨子 → 变址**；**立级 → 立即**；**木 → MOV、铺石 → PUSH、字针 / 字件 → 自增 / 自减、2MUL → IMUL、案的货 → AND / 逻辑与或、Core → CALL、Return → RET**；MA2 / MAL → MAR、MD2 / Mdl / MD20 → MDR、CO → CU、i2 / I2 / AR → IR；**时中周期 → 时钟周期、智长 / 自责 / 资厂 / 制裁 / 自成 → 字长、存储自藏 / 村众智长 → 存储字长、指定智长 → 指令字长**；**编织 / 编子 / 编审 / 编折 → 编址**；**二净字 / 二金值 / 二进字 → 二进制、十六镜子 / 16 斤制 / 16 进字 → 十六进制**；**补马 → 补码、圆满 → 原码、帧值 / 曲值 → 真值 / 取值**；**刺高位 → 次高位、敬畏 / 经位 → 进位、界位 → 借位**；**敷点数 / 辅典数 → 浮点数、接码 → 阶码**；**薛格 / 苏格 / SUB → SUB、C 应 / C 硬 → C_{in}、C2 / C out → C_{out}**；**货运算 → 或运算、一位器 → 移位器、自争 → 自增**；**内村 → 内存、手地址 / 手地值 → 首地址、片一辆 / 偏一亮 / 二层偏一辆 → （二乘）偏移量**；**天堂/相临 → 大概/相邻、客家/刻下 → 课下、荒 → 慌**。

数字类与拼不回来的地方不在本表展开，逐条列在 §附的「转写含糊处」清单里（含讲稿把 `1FD4` 转成「EFD4」、把 `2100H` 转成「2000 亿」两处硬伤）。

1 考情分析：第四章的分量与做题观

「今天的话主要看一下（计算机）**组成原理里的第四章**。」「相对来说，组成原理的大题考的还是比较（集中）的。」「大概**考大题的只有八个地方**，而且这八个地方**有的地方考的多有的地方考的少**。」「这个第四章的话……**也是一个（大题）的高频考点**。」

「第四章指令，其实这边第四章**虽然只有四道**（大题），但是其实到后面的……像流水线，有的题目**本质上也是考指令**，包括这个数据通路，有的地方也是和这个第四章相关的，**就是组成原理的四五章是有关联的**」「（一、二、三章）其实没啥关联……**四五章往往是有关联**：像这个指令、汇编、数据通路、流水线，这些都是有关联的。」

他给的复习观：这一章「书薄、题活」

「（第四章）典型的就是**知识点很少**」「你看书其实看得很快，书上内容不多，**但是要通过题目去练习**。」「第四（章）多做题」「做完题目之后还要把知识点再（去）复习复习。」「（有）的时候做题目会发现**很多题目和书上都是不一样的，比较灵活**，但是也还好，都不算太难。」

「汇编这一块……408 组成原理（的）汇编它其实是两门课（的东西），（4）08 的话，汇编就会用到一点点，**但是不会考得特别深，简单了解就可以了**，通过这一个小节就够了，**不用花大量的时间去研究汇编**，就把这些画的记一下就行。」

为什么必须把真题做大题做到透：「408 的大题它的**出题质量很高**，市面上没有什么模拟题能达到这个（大题）的出题质量」「所以就是说一定要把这些真题**完全搞透**。」「这个 408 的真题它经常有时候会**根据以往的一些大题重新出一道新的大题**，所以你只有说把往年的大题搞得特别透、做的特别熟，**即便考试碰到一个新的，你也能很容易去联想到（旧）的题目怎么考的**。」「（计算机）组成原理这门课比较特殊，**它考的都很灵活**……（数据结构、操作）系统或者（其他），可能考的（大多数）考法还是比较固定的，**组（原）的考法是最灵活的，尤其是指令和 CPU 这里**。」

「讲过的大题**千万不要说只做一遍**：讲过的大题课下立马做一遍，然后隔个三五天……哪怕没有时间动手算，你把那个思路捋一下」「一般来说这个要到**第二遍或者第三遍**，可能才会有这个熟练度。」

他顺带给的两条考场策略（本讲唯一提到分值/时间的地方）

「考试的话，**选择题 40 个**，基本是要控制在 **60 到 70 分钟**完成」「你如果做的再慢，大题时间就不够了……很容易慌。一定要给大题预留充分的时间。」「（选择题）一个就两分，碰到一些不太确定的，可以先选一个，先把它标记一下，做完大题再来检查就可以了。」

「（一个大题分了四小问，每一小问又分成好几个小问）所以其实**每（小问的）分都很少**，基本就是一个小问**一分或者两分**」「所以一般来说其实**不会太看你的计算过程**，考试的话不要花太多时间去写过程，**基本没有过程分**，这个答案对了就对了，错了就错了。」「除非……你可以把过程写得详细一点，但大多数都是只写答案就行。」

但他补了一句反话：「像这种题你看着很复杂，**但是反而是很容易拿满分的**——如果真正学完学会之后、搞懂之后，**每一小问他都是有理有据，都是能分析出来的**。」「**题干的每一个条件其实也会发现基本都是有用的**。」

本讲实际覆盖的块面（对照你给的大纲）：① 指令格式（操作码 / 地址码、0～4 地址）——**讲了**；**扩展操作码——本讲未讲（全讲 0 次，详见 §8 边界）**。② 寻址方式与 EA 计算——**讲了，且是主体**（十种，比大纲列的多出**隐含寻址、堆栈寻址**）。③ **指令周期 / 机器周期——本讲未讲（0 次）**。④ 汇编语言与机器级表示——**讲了**（指令表 + 四道大题，课内精讲两道）。⑤ 真题——提到了 **2015 / 2017 / 2018 / 2019 / 2023** 年，但**课上精讲的那几道他都没说年份**（§5）。

2 核心概念（含第一章回顾）

2.1 第一章回顾之一：MAR / MDR 与三条总线

> 「**MAR 是用来反映存储单元的个数**」「**MDR 等于存储字长**」「只要题目问你 MAR 的位数，其实 **MAR 的位数就等于地址线的位数，其实就等于总共的地址数量（取 $\log_2$）**」「**MDR 的位数往往就等于数据线的位数，又等于存储字长的位数。**」「这是一定要重点留意 MAR 和 MDR。」

2.2 第一章回顾之二：编址方式（按字节 / 按字）

> 「假设说编（址）方式，一个是**按字节编址**，还有一个就**按字编址**」「**这个按字编址的这个「字」就是存储字长**」「（一）个字节是 8 个比特」「如果是按字节编址，就**一个字节占一个地址**」「其实**编址方式它只是一种规范、一种规则**」「你无论怎么编，**总共内存都是十六个字节，这个是（不会变的）**，只不过说一些规则、编（地）址的规则不太一样。」
> 「（有的题目就会问）让你去分析**它是按字节编址还是按字编址**，一定要注意这一点。」

编址方式	一个地址有多大	他举的例（同样 16 字节内存）	地址数量
按字节编址	1 字节（= 8 比特）	存储单元 32 位 → 每单元占 **4 个地址**	16 个地址，每地址 1 字节
按字编址	1 个字（= 存储字长，16 / 32 位都可能有）	一个字直接占 1 个地址	4 个地址，每地址 4 字节

> ⚠ **他反复警告的一条：碰到「按字编址」要小心**
>
> 「**凡是做题目碰到按字编（址）一定要小心一点**，因为 **80% 的题目都是按字节**（编址），如果突然出现按字编址，要注意一下，看一下有没有陷阱。」
> 「只要是按字编（址），**一定得告诉你每个字多大**，不然题目就没办法做了。」「如果（题干上）没有这个信息，那就只能（拿）计算机字长来算。」

2.3 第一章回顾之三：三个「字长」与 CPU 执行时间

> 「通常说的 16 位或者 32 位机器，它指的是**机器字长，简称字长**，指的是**计算机能处理的二进制的位数**」「机器字长是用来**反映每次运算、一次运算能处理的位数**」「（通常）和寄存器、ALU 有关」「其实最后要记得就是**机器字长这个名称，包括简称**——其实（机器字长）和通用寄存器（的位数）和 ALU（的位数）相等。」

> 「（一条）指令的长度，这个位数叫**指令字长**」「**存储字长就是一个存储单元的位数**」「这三个在真题里面经常会出现的，要区分开。」

```
机器字长  = 一次运算能处理的位数 = 通用寄存器位数 = ALU 位数（问 ALU 位数 → 就是在问机器字长）
指令字长  = 一条指令的长度（题目说「指令寄存器 16 位」「指令 16 位」→ 指令字长）
存储字长  = 一个存储单元的位数 = MDR 位数 = 数据线位数
```

> **顺带过的时序与性能（他原话：「整个就了解这些就可以了」「理解之后就很好记，这不需要硬记」）**：「时钟周期，**计算机最小的执行时间单位**」「时钟频率，就**时钟周期的倒数**」「CPI，就**执行一条指令需要的时钟（周期数）**」「CPU 的执行时间 = **指令（条数）× CPI ÷ 主频**。」

2.4 指令格式：操作码 + 地址码；0～4 地址指令

> 「（指令）分为**操作码和地址码**，这些都是比较简单的」「**操作码是用来反映这个指令执行什么操作**」「**地址码是用来给出这个数据的地址**」「指令通常是**字节的整数倍**。」

```
零地址：只有操作码没有地址——「关机、中断、空操作，都不需要地址码」；也可（弹出）两个操作数
一地址：「一种是只有一个目的操作数」；另一种「指令提供一个操作数，另外一个由 ACC 提供」
        ⚠️「一地址并不是说一定只有一个数据，有可能是两个」
二地址：A1、A2 都是地址；「运算之后到底写回到 A1 还是 A2，这个并不是固定的」→ 看题目要求
三地址：前两个地址的数据运算后写回第三个地址；「有的题目会把目的操作数写回到最前面，都是比较灵活的」
四地址：多一个 A4，用来「表示下一条要执行的指令的地址」；「这些都了解就可以了」
```

2.5 指令寻址 vs 数据寻址；以及「括号」这套记号

> 「（寻址方式）分为两大类：一个是**指令的寻址**，还有一个叫**数据的寻址**」「你无论怎么寻，它都是要通过 PC，因为 PC 放的就是指令的地址。」
> **顺序寻址：**「你让 PC 加一，自动形成下条指令的地址」「当然你并不是一定加一：**如果按字节编址，一条指令占两个字节，那就是（一条指令）占两个地址，那 PC 是要加二的**；如果一条指令占四个字节，那就要加四。**PC 加几，就看一条指令占几个地址。**」
> **跳跃寻址：**「就是一些转移指令」「**它也是通过修改 PC 的内容**」「所以（书上）这个指令寻址你看篇幅不大，就两段话……**无论怎么弄都是修改 PC。**」

```
记号规范（他专门停下来讲的）
  A      = 形式地址（地址码里那个数）
  (A)    = A 这个地址里的【内容】；若 A 是寄存器编号，(A) 就是寄存器的内容
  EA     = 有效地址（操作数真正的存放地址）
  间接寻址：EA = (A)      操作数 X = ((A))      ← 加一层括号 = 取一次地址
  寄存器间接：EA = (R)    操作数 X = ((R))
```

> 「注意这种写法：**如果 A 是一个寄存器，你（写）括号 A 表示这个寄存器的内容；如果 A 是一个内存（单元），你（写）括号表示这个内存里面的内容**」「A 是一个地址，如果再加个括号，就表示把这个地址的数据取出来。」
> 「（今后）做题目以后经常出现这种加一个括号和加两个括号的。」
> 「用 A 表示形式地址，就是地址码里面的这个（叫形式地址）；然后**最终操作数的真实地址叫有效地址，叫 EA**。」

2.6 数据寻址十种：EA 公式 + 他的原话（最该背的一张表）

方式	EA	他举的例子 / 原话	备注
隐含寻址	—（操作数在 ACC）	「（地址码）是一个 10，它就会找到内存的 10 号单元……假如说 10 号单元放了一个 X，现在一地址的话，你已经把第一个数据 X 取出来了」「这个 ACC 这个累加器可以（额外）提供另外一个数……ACC 里面放了一个 Y」「**明明指令里面只有一个地址，但是可以把两个数据运算**」	第二个操作数由 ACC 隐含提供
立即寻址	无（数在指令里）	「你要把一个数据 10 去进行运算，直接把 10 这个（数）本身放进去」「**地址码放的不是地址而是数据**」	取数最快，范围受指令字长限
直接寻址	EA = A	「（地址码）这边放了一个地址 10，他就找到内存的 10 号，10 号里面放了一个 X，这样就能把 X 取出来」「**指令里面提供的就是内存地址**」	一次访存
间接寻址	EA = (A)	「指令的地址码给的不是真正的地址，比方说这里是一个 10，它找到内存的 10 号，10 号里面又放了一个数字 100，**就告诉他还要再去找内存的 100 号地址**」	两次访存；A 是形式地址
寄存器寻址	EA = R（数在 R 中）	「你（直接）提供一个寄存器编号是 4……**那 4 号里面的东西就是 X**」「这个 R、这个寄存器的编号它就是形式地址」	不访存，最快
寄存器间接	EA = (R)	「先通过寄存器的编号找到了一个 4 号寄存器，**4 号寄存器里面又放了一个内存地址**，比方说这里放了一个内存地址是 100，然后再去找到内存的 100 号再去取 X」	一次访存；写成 ((R))
相对寻址	书上：EA = PC + A	「让这个 PC 加上一个 A，A 的话往往叫做一个偏移量」「PC 里面是一个 100，当前这个形式地址是一个 2，那计算机就会把这个 100 和 2 相加得到一个 102，最终去找到内存的 102 号取出了数据 X」	**广泛用于转移指令**；**真题算式不同，见 §3.3**
基址寻址	EA = BR + A	「（用）基址寄存器的内容加上指令的形式地址，然后形成有效地址」「基址寄存器**既可以采用专用的寄存器，也可以指定（用）一个通用寄存器**」	**用于程序浮动**（多道程序重定位）
变址寻址	EA = IX + A	「（让）变址寄存器的内容加上形式地址」「变址寄存器也可以专用的也可以通用的；这个书上图只画了一个专用的」	**主要用于处理数组问题**

方式	EA	他举的例子 / 原话	备注
堆栈寻址	栈顶指针隐含	「（操作）系统里面有一个栈，里面放了一堆数据，你需要用的时候往外（弹）、往外出栈，出两个元素就可以了」「**这个也是了解即可**」	他说不重要

> 课尾他自己做的小结（本讲最该背的一句）
>
> 「所以这边寻址方式……真正需要注意区分的就是这几个：**相对（寻址）是 PC 加上（偏移量），基址（寻址）是 BR 加上（偏移量），变址（寻址）是 IX 加上 A。**」
> 「然后**基址寻址的话，这个 BR 它的内容不变，你即便执行多条指令它也不变**；**然后变址寻址的话，每条指令这个 IX 的内容都在变，但是 A 是不变的。**」

2.7 基址 vs 变址：谁在变、谁不变，以及各自的应用场景

```
他现场画的两组对照（都是 3 条指令、地址码分别是 0、1、2）
【基址寻址】BR = 100（不变）          地址码 = 0 / 1 / 2（在变）
   第 1 条：100 + 0 = 100  → 取内存 100 号
   第 2 条：100 + 1 = 101  → 取内存 101 号
   第 3 条：100 + 2 = 102  → 取内存 102 号
   ⇒ 「在多条指令的执行过程中，BR 的内容是不变的」

【变址寻址】IX = 0 → 1 → 2（在变）   地址码 A 不变
   第 1 条：0 + A        第 2 条：1 + A        第 3 条：2 + A
   ⇒ 「变址（寻址）就是寄存器在变，指令不变」
```

基址 → 程序浮动（他把进程挪了个位置）：「假如说一开始这个内存有一个进程，它用了这个 100 号到 199 号」「这时候你只需要把这个**基址寄存器设置成 100**」「（100 条指令）分别会去取内存的 100、100 加 0、100 加 1、100 加 2，一直到 100 加 99」
「现在你突然把这个进程的数据**挪到了 1000**（到 1099）的位置，那现在其实你**只需要改一下这个基址寄存器的内容就行了**：把 100 改成 1000」「这样的话你就会取到 1000、1001、1002，一直到 1099」「（把）一些这个进程的数据去移动的时候，**只需要改一个基址寄存器，就很方便**。这个叫**便于程序浮动**。」

变址 → 数组（他直接翻了个真题选择题）：「这个变址（寻址）主要是用作这个数组」「**主要是处理数组问题**」「（数组的）每一个下标的地址就可以这样去算：首地址加上下标乘以大小。」

2.8 寻址特征字段：为什么指令里要多出这一小块

「（前面已经讲了 6 个寻址方式了）所以平常来说你指令是只有操作码和地址码，（现在）会专门多了一个东西叫**寻址特征**」「**寻址特征用来表示你用的是哪一个寻址方式**」「因为你不同的寻址方式，这个流程是不一样的：比如说（寄存器寻址），你寄存器直接寻址还是寄存器间接寻址，**它都是后面放了一个寄存器的编号**，那计算机内部他怎么（知道）……**肯定要看寻址方式的**」「当然这个要**灵活判断，要根据具体的题目再来看他是否需要**，但是要知道有这么个东西。」

2.9 汇编指令速览（书上 184–185 页那张表，他带着画了一遍）

```
数据传送：MOV（他念「木」）   传数：寄存器 / 内存 / 常数 → 寄存器或内存
          PUSH（「铺石」）入栈，常用于参数调用；POP 出栈
          ⚠「栈增长方向与内存地址增长方向相反」——入栈时栈顶指针【减小】，出栈【增大】
算术运算：ADD / SUB（加、减）    INC / DEC（「字针和字件」= 自增 / 自减）
          IMUL（转写成「2MUL」）有符号数乘法
逻辑运算：AND / OR / NOT / XOR（「与、或、非、异或」）    SHL / SHR（左移 / 右移）
控制流：   JMP（无条件转移，直接跳到最终地址）
          条件转移 Jcc：JE（相等）/ JZ（结果=0）/ JNE（不等）
                        JG（大于）/ JGE（≥）/ JL（小于）/ JLE（≤）
          CMP（比较——「计算机内部先让它做减法」）
          CALL（函数调用，他念「Core」）/ RET（返回，他念「Return」）
```

> 「这个控制流指令要记下这个转移指令，这个真题也是出现过的、考过的」「这个条件转移你要记一下后面的**英文全称**，相当你要把缩写和全称要能对应起来。」「**JMP 是无条件转移**」「这个 CMP（是）比较」「这个 CALL 和 RET 也要看：**CALL 是函数调用，RET 是返回**」「重点就看画的那些就够了。」
> 「**即便到了二三年，也是有考察过的。**」

> ⚠ **他只讲了「哪些要记」，没有展开每条指令的编码**
>
> 全讲**没有出现「扩展操作码」「指令周期」「机器周期」「取指周期」「间址周期」「执行周期」「微操作 / 微指令」中的任何一个（逐字检索 0 次）**。他讲汇编指令时只到「缩写 ↔ 全称 ↔ 大致功能」这一层，**没有讲各条指令的操作码编码**（唯一出现的编码是那两道大题题干里给的）。本资料不替他补。

3 解题方法论：五套判断与算式

3.1 怎么判编址方式（三条线索，本讲出现两次）

```
线索①【PC 加几】——最常用
    "PC 加 2" ⇒ 一条指令占 2 个地址；一条指令又是 16 位 = 2 字节
              ⇒ 一个地址 = 1 个字节 ⇒ 【按字节编址】
    反推：如果题目把 PC 加 1（一条指令 2 字节却只占 1 个地址）
              ⇒ 一个地址 = 2 字节 ⇒ 【按字编址】，且这个字 = 16 位
线索②【地址差 ÷ 字节数】——他做「机器代码占多少字节」那道题用的
    末地址 − 首地址 + 1 = 地址个数；再 × 每个地址的大小 = 字节数
线索③【题干直接给】「按字节编址」「每（个地址）……」
    ⚠ 给不出线索时不要自己假设：他的原话是「到底它（指）是哪个意思，你可能需要具体到题目去分析」
```

> 「（书上）经常会讲 PC 自动加一，**这个「一」的话是一条指令占几个地址**」「现在这个，它为什么加二，那就说明一条指令占两个地址」「那一条指令又刚好是两个字节，那说明**一个地址就是一个字节，那就是按字节编址**。」
> 「如果把 PC（改）加上加一，你就是**按字编址**：因为一个指令明明是两个字（节），但是你只给它加了一个地址，那说明一个地址就占两个字（节）……按字编（址）的话，它 16 位才用了一个地址。」

3.2 三个「字长」怎么套：缺哪个就用哪个

```
问 ALU 位数 / 通用寄存器位数  → 机器字长（题目里的「计算机字长」也常指它）
给【数据线位数】/ 说【存储单元大小】→ 存储字长 = MDR 位数
说【指令 16 位】/【指令寄存器 16 位】→ 指令字长
⚠ 题目只说「字长」时，先看缺哪个：他原话「你只要能把机器字长、指令字长、存储字长三个概念
   搞明白，到时候你看到缺哪个（就）当中（用）哪个」
```

「他这里专门说计算机的字长……我们刚才讲过字长就那几个：存储字长、指令字长、机器字长」「对于这个题目来说，**这个「字长」指的是存储字长**，不然题目没法做了」「所以**到底他表示什么意思，你得看题目去分析。**」「（另一道题）明确告诉你了这个数据线的位数……**数据线的位数就是一个存储单元的大小，也是 MDR 的大小**」「然后这个 16 位是指的**指令字长**」「唯独这个题目机器字长没法算了，没有告诉你，所以这里（的「计算机字长」）指的就是机器字长。」

3.3 相对寻址在真题里的算式：PC + 指令长度 + 系数 × 偏移量

```
书上的模型          EA = PC + A
真题的口径          目标地址 = (PC) + 指令长度 + 系数 × A      ← 顺序是「先取到下一条指令地址，再加偏移量」
本讲两处实例（系数都是 2，因为偏移量单位是「字/半字」而不是「字节」）：
   第七题(175 页)：200C + 2 + 2 × (−29) = 1FD4
   第二题(阶乘题干)：00401025 + 5 + (−42) = 00401000

【向后跳转能跳多少条】偏移量是 8 位补码 → 取值范围 −128 ~ +127
   A = −1  ⇒ 原地踏步（0×2+2 = 2，PC 恰好回到自己）
   A = 0 及以上 ⇒ 向前（往地址大的方向）
   A = −128 ~ −2 ⇒ 向后，条数 = (−2) − (−128) + 1 = 127 条
   ⚠ 区间里有多少个数，一律「末 − 首 + 1」
```

「正因为……书上（学的时候）可能就是直接 PC 加上一个 A，**（但）在做题目的时候它都是这样的：先用 PC 加（上指令长度）——PC 要先取到下一条指令的地址，然后再去加偏移量**」「然后还要注意**偏移量到底有没有系数**」「（题目里）这个 A 还会加上一些系数，包括这个 PC 它还会加上一个 2，**所以就是和书上那些要区分开。**」「**如果顺序执行的话，PC 是直接加 2**」「（转移时）就是 PC 加 2 再加上二倍的偏移量。」「（问 3 到 6 有几个数）一定是 6 减 3 加 1，4 个数」「问你一个区间 A 到 B 有多少个数，**一定是减完之后要加 1 的。**」

⚠ **本讲最容易翻车的一步：无符号的 PC 与有符号的偏移量不能直接加**

「你指令的地址一定是无符号数，前面一定是无符号数，后面（偏移量）就变成有符号了」「这个千万不要这样做，这边有一个误区：有的人可能就是说 200C 加上一个 2……偏移量又是一个 E3，然后有人可能会把这个 E3 直接去通过（符号位）进行（扩展）相加——**不要直接去加**」「**你把一个无符号数和有符号数是不能直接参与运算的**」「你一定要把它的真值先求出来，你求出来是一个 −29（×2 之后是 −58），其实就相当是 200C 减去一个（0x38）。」

3.4 变址取数组元素：A + i × 元素大小（真题比书上多一个「乘」）

```
数组元素地址 = 形式地址 A（= 数组首地址） + 变址寄存器内容 i × 元素大小
   按字节编址、元素占 4 字节：A + i×4（首地址 = A + 0×4，下标 1 = A + 1×4，下标 2 = A + 2×4）
```

本讲那道题是 double：元素占 8 个地址 → A + i×8

⚠ 真题的「变址寄存器」里放的是【数组下标】，不是字节偏移：
「注意变址寄存器存放的是数组下标，专门告诉你了」「每次用这个下标还要乘以一个大小」
所以题干只要写了「按元素大小偏移」，就必须乘，不能照抄书上的 IX + A

「所以这时候和书上讲的不是太一样：书上当时讲变址寻址，你就是**让寄存器加上偏移量就可以了**」「所以实际上你在做题的时候，有时候**这个变址寄存器可能还要乘上一个东西，乘上一个东西再加上这个指令里面的形式地址**」「这个题目就（是）变址寄存器的内容**就要乘以 double 的大小**。」
「相当这个下标不停加一，每次还要让……某一次变址寄存器的下标是 i，那就应该用这个指令里面这个形式地址**加上变址寄存器的内容、再乘以大小**，因为题干（说）要乘以大小。」

3.5 CF / OF 的两条路：观察法（做选择）与公式法（计算机内部）

【路一 | 观察法】给选择题用，事后角度看
小减大 → 有借位 → CF = 1 ；大减小或两数相等 → 无借位 → CF = 0
想判「相等」还要配 ZF：CF = 0 且 ZF = 1 ⇒ a = b
CF = 0 且 ZF = 0（结果非零）⇒ a > b

【路二 | 公式法 + 加法器】计算机内部就是这么算的
减法：SUB = 1 → 第二个操作数【全部取反】+ 最低位进位 Cin = 1 → 转成加法
CF = SUB ⊕ Cout　　OF = Cn ⊕ Cn−1　　（公式在书上第二章加法器那一节）
例（本讲第一题）：i = 0，n−1 = 0xFFFFFFFF
0 + ~(0xFFFFFFFF) + 1 = 0 + 0 + 1 = 1　⇒ Cout = 0、Cn−1 = 0
CF = 1 ⊕ 0 = 1　　OF = 0 ⊕ 0 = 0

「计算机（在比较两个数时）……**不会说先知道（谁大谁小）再去算 CF**」「CF 它可以表示进位，也可以表示借位：**如果是加法的话表示进位，如果是减法的话表示借位**」「那 CF 等于 1 的话，就说明是有借位，那为什么有借位呢？说明不够减——小的减大的」「然后如果 CF 等于 0，你单纯通过一个 CF 等于 0，它只知道没有借位……**为了更进一步区分，你还要借助于 ZF**。」
「所以（无符号数）小于的话 CF 等于 1；**如果是等于的话，要 CF 等于 0、ZF 等于 1**」「**但是注意这种方法只是你去做选择题可以这样用，计算机内部肯定不是这样算的。**」「（考试时）你就说小减大，直接算出 CF 等于 1……答案是正确的。」

⚠ **他专门纠的一个答法错误**

「**千万不要去回答 CF 的数值和 ZF 的数值，人家问的是检测位**」「你千万不要根据 CF（的）数字本身去填空去写这个答案，你要理解题干上这个**检测位**的含义。」「如果这个题目更进一步问「然后问你 CF 和 ZF 的具体的取值是多少」，你也应该（会）回答：CF 等于 1（或 0）、ZF 等于 1。」

3.6 C 代码 → 汇编 → 机器码：答题五步

① 读题面：这类题都是「C 语言（高级语言）+ 行号 + 虚拟地址 + 机器指令（16 进制）+ 汇编」五列并排
② 抓操作码：题干会给助记符对应的编码（例：ADD = 0010）
③ 判每个操作数的寻址方式 —— 只看有没有括号：
R4　　→ 寄存器（直接）
(R4)　　→ 寄存器【间接】（寄存器里放的是地址）
((R4))　→ 再取一层

④ 按指令格式逐字段拼二进制，每 4 位一组转十六进制
例：0010 | 001 | 100 | 010 | 101 = 2315H
⑤ 执行结果：各按寻址方式取一个操作数 → 运算 →【写回目的操作数（= 右边的那个）】
⚠ 别漏隐含动作：「这个寄存器还有一个自增」，问「哪些寄存器/存储单元会变」要一起写

「今后做题目里面，**即便没有题干上告诉你，也得能看出来**」「正常来说一条指令应该有操作码和地址码……那既然你（看到）的地址码 R4 加括号，**那说明寄存器里面的内容才是地址**；你如果是寄存器直接（寻址），那就不应该加括号」「**只有说寄存器间接（寻址），寄存器里面的内容它是一个地址。**」
「**目的操作数的意思就是你把两个数进行运算，最后还是写回了右边的这个寄存器**，因为它叫目的操作数。」

3.7 做「多小问大题」的两条经验

「你在做这几道大题的时候可以考虑一个问题：**不同的小问之间，看是递进的还是说毫无关联（的），都有**」「如果是那种递进的，它往往可能第一问比较简单，你前面小问如果做好的话，后面自然就做出来了」；**没关联的，某一个不会就跳过去。**」
以及「猜出题人」：「**同一个问题在出试卷的时候往往不会连着出两次**，所以你读完题应该大概就知道后面那个一定和前面不太一样……（如果）前面那个考的是转移执行，那后面大概率考的就是顺序执行。」

4 典型题型与例题（他课上手推的六道）

下表题号是**他手上讲义 / 课后题的编号**（16 题、第七题、第一题、第二题、课后第九题、以及一个真题选择题），**不是统考年份**。他本讲**亲口说出的年份集**中在 §5。

4.1 变址取数组元素（真题选择题）：变址寄存器内容 = 32

题干要素：数组首地址 = 2000H；数组元素是 double，每个占 8 个地址；按字节编址
变址寄存器里放的是【数组下标】；某一次取出的元素地址 = 首地址 + i × 8

解：8i = 0x100 = 16 × 16 = 256 ⇒ i = 256 ÷ 8 = 32
★ 本资料独立验算：256 ÷ 8 = 32 ✓
⚠ 讲稿这一段的元素地址被转写成「2000（亿）」，与「8i = 0x100」互相矛盾：
若元素地址真是 2000H，则 8i = 0、i = 0，和他的算式不符。
按他自己的方程，元素地址应为 2100H（相差 0x100）——本资料只保留
「首地址 2000H」与「i = 32」这两个能自洽的量，不替他补第三个。

「注意变址寄存器存放的是数组下标，专门告诉你了，**保存数组下标**」「一开始下标是 0」「每次取完一个数据，**变址寄存器的内容会自动加一**」
「现在他说某一次……取了元素的这个地址（是 2100H），那就是首地址加上这个当前变址寄存器的内容，再乘以这个大小」「**把 i 解出来就可以了**」「这样解出来是一个 8i 等于（0x100），所以这里是十六进制的一百」「**十六进制的一百就是十六乘十六**」「最后 i 就等于这个三十二。」
他借这道题说的真题规律（本讲最值钱的一句年份信息之一）：「除此之外，**18 年为什么专门考了变址寻址**？这样的（题）在真题里面 **17 年也是考的**——他问「适合按下标访问一维数组的是哪一个」，**那就是变址寻址。**可能就那一年这一个题答得不太好，可能下年又出了一道这样的题，**相邻了两年。**」「所以 408 一定要把真题……**真题一定要多做几遍**，做的时候一定要多思考一下，千万不要说选完一个选项就过去了。」

4.2 大题①（课后 16 题）：ADD (R4), (R5) 的机器码与执行结果

```
题干要素：ADD 的操作码（助记符）题干给了 = 0010；操作数写作 R4、(R5)
          R4 的内容 = 1234H，R5 的内容 = 5678H
          内存 1234H 单元放的是 5678H，内存 5678H 单元放的是 1234H

(1) 写出机器码（寻址方式位 + 寄存器号，各 3 位，拼成 16 位）
    0010 001 100 010 101      →  每 4 位一组：0010 0011 0001 0101 = 2315H
    操作码 ADD | 方式 001=R4 间接 | R=100(4) | 方式 010=R5 间接 | R=101(5)
    ★ 本资料独立验算：0010 0011 0001 0101 = 0x2315 ✓（与他念的 2315 完全一致）

(2) 两个操作数各取什么
    (R4)：R4 = 1234H 是地址 → 内存 1234H 的内容 = 5678H   → x = 5678H
    (R5)：R5 = 5678H 是地址 → 内存 5678H 的内容 = 1234H   → y = 1234H

(3) 执行结果与「哪些会变」
    x + y = 5678H + 1234H = 68ACH   （逐位：8+4=12=C；7+3=10=A；6+2=8；5+1=6）
    ★ 本资料独立验算 0x5678 + 0x1234 = 0x68AC ✓
    目的操作数 = 右边的 (R5) → 结果写回【内存 5678H 单元】：1234H → 68ACH
    R5 另外还要自增：5678H → 5679H    ★ 独立验算 ✓
    ⚠ 讲稿在这一问转写成「哪些寄存器会变的就是 101 号、5 号寄存器会变」——
      按他自己的分析（R4 全程只被读、结果写回 R5 指向的单元），只有 R5 会变。
```

「既然题干上专门告诉你这个加法（助记符）是 0010，那你就把操作码写一个 0010」「然后现在的话这个 R4 加括号，括号的话是一个这里叫**寄存器间接寻址**，他的寻址方式位应该是 001」「第二个地址的话是括号（R5），它是一种叫间接寻址……**他的寻址方式就应该是 010**」「用 16 进制表示的就每 4（位一）组……相当（求出）是 2315，**机器码是 2315**。」

「（执行）你把这两个数进行运算，16 进制可以直接运算」「最后算出来是一个 68AC」「**因为目的操作数是右边这个，所以你最后算完这个结果会写回到右边这个地方**」「除此之外，这个寄存器还有一个**自增**」「这个寄存器本来是 5678，你自增完之后就变成了 5679。」

4.3 课后题：MAR / MDR 位数与主存地址空间（含「改成按字编址」的变式）

```
原题：机器字长 16 位；主存地址空间 2^20 个地址（地址线 20 位）；按字节编址
   (1) ALU 位数 = 机器字长 = 16 位          （「问 ALU 的位数，那就必须知道机器字长」）
   (2) 主存地址空间大小 = 2^20 × 1B = 1MB   （他念「一兆」）★ 独立验算 2^20 = 1048576 = 1MB ✓
   (3) MAR = 地址线位数 = 20 位；MDR = 数据线位数

变式（他当场改题）：改成按字编址、数据线 32 位
   2^20 个地址不变，但一个地址 = 一个字 = 32 位 = 4 字节
   → 地址空间 = 2^20 × 4B = 4MB（本资料按其口径补算，他未念出结果）

另一题：内存 128KB、按字编址、计算机字长 16 位（当作存储字长）
   → 每个字 2B → 地址数 = 128KB ÷ 2B = 64K = 2^16 → MAR = 16 位、MDR = 存储字长 = 16 位
   ★ 独立验算 128×1024 ÷ 2 = 65536 = 2^16 ✓

同题其余小问（不需要算，数位数就行）：
   操作码 4 位 → 最多 2^4 = 16 条指令
   寻址方式字段 3 位 → 寄存器号也 3 位 → 2^3 = 8 个寄存器（反过来：给 16 个寄存器 → 4 位）
```

> 「凡是问你**指令系统能表示多少条指令**的，你就只看操作码的位数就行了，**指令的条数只和操作码有关**」「（寄存器号）占三位，它就能表示 8 个寄存器」「（反过来）他就说寄存器有 16 个，那寄存器（号）就是占四位。」「**这种问题在 408 里面反复考**……**很多 408 的题目就喜欢正反就考**」——他随手举了三处：块大小 32 字节 ↔ 块内地址 5 位；页面 4KB ↔ 页内地址 12 位；8 个寄存器 ↔ 寄存器号 3 位。

4.4 第七题（讲义 175 页）：条件转移指令 —— 从 200C 跳到 1FD4

```
题干要素：指令 16 位、按字节编址；条件转移指令格式 = 前 5 位操作码 + 3 位检测位 CZN + 8 位偏移量
          「检测位只要有一个 1 就转移，否则不转移」；转移时 目标 = PC + 2 + 2 × 偏移量

第一小问（1）按字节还是按字编址？→ 按字节（PC + 2；一条指令 16 位 = 2 字节 → 一个地址 1 字节）
        （2）「这个指令向后跳转（多少）条」→ 偏移量是 8 位补码
             取值范围 −128 ~ +127；−1 原地踏步；0 及以上向前
             向后可用范围 −128 ~ −2 → 条数 = (−2) − (−128) + 1 = 127 条   ★ 独立验算 ✓
第二小问（1）CZN = 011、当前指令地址 200C、偏移量字节 = E3 → 011 只看 ZF 和 NF，
             恰好有一个为 1 ⇒ 转移：目标 = 200C + 2 + 2 × (−29) = 200E − 0x3A = 1FD4H
             等价算法（他自己用的）：0x200C − 0x38 = 0x1FD4
             逐位：C−8 = 4；0−3 借位后 16−3 = 13 = D；被借的 0 变 15 = F；2 被借 1 位 → 1
             ⚠ 讲稿把结果转写成「EFD4」——两条独立路线都只能得 1FD4H，「1F」被转成「EF」
        （2）同一小问的另一空：CZN 仍是 011，但 ZF 与 NF 都为 0 ⇒ 不转移 ⇒ 顺序执行
             200C + 2 = 200E（他顺带讲：出题人不会连着出两个一样的，所以这一空一定换口味）
第三小问：无符号数比较「小于等于」要检测哪几位？
             小于 ⇒ CF = 1 ；等于 ⇒ CF = 0 且 ZF = 1 ⇒ 只用到 CF 和 ZF ⇒ 检测位 = 110
             ★ 独立验算：C = 1 检测 CF、Z = 1 检测 ZF、N = 0 不用 → 110 ✓（他念的也是 110）
第四小问：给一张数据通路图，写 1、2、3 的名称或功能（「名字真忘了，记得他是功能也可以」）
             1 = 指令寄存器 IR（「既能放操作码，又能放（偏移量），明显是上面这个指令」）
             2 = 移位器（把 offset 乘 2）    3 = 加法器（把 PC+2 与 2×offset 相加）
             路径上还有一段符号扩展：把 4 位有符号数变成 8 位有符号数（「数字本身是没有变的」）
```

> 「注意在真题里面，凡是转移指令、就是那个相对寻（址）……（题目里）都是这样的：**先用 PC 加（指令长度），PC 要先取到下一条指令的地址，然后再去加偏移量**」「**至于为什么要加二**……因为一条指令占两个字节。」「（偏移量的）取值范围：8 位补码的表示范围应该是 −2 的 7 次方到 2 的 7 次方减一」「（−128 到 127）」「**你想要向后偏移，它就只能是这个 −2 到（−128）**」「**从负 128 到负 2 都是向后的**；负 1 就不行了，负 1 就已经原地踏步了；零（及以上）就正向（偏移）了」「那总共就是……**最后就 127 条**。」
> 「（转移那一问）那你就用 PC 200C 加上一个二，再加上这个（二倍的）偏移量」「**这个地方（偏移量）是一个 E3**」「你一定要把它的真值先求出来，你求出来是一个 −（29），（乘 2 之后）其实就相当是 200C 减去一个（0x38）」。

> ⚠ **这道题里他念错、随后自己改口的一处（本资料按改口后的值）**
>
> 他在推「这个指令向后偏移多少条」时把 8 位补码范围念成「**−128 到 −127**」——**上限应为 +127**（同一句里他给的公式「2 的 7 次方减一」就是 127）。这是**老师口误**，本资料按 **−128 ~ +127** 呈现，并据此得到「向后可用 −128 ~ −2，共 127 条」（他念出的 127 条是对的）。
>
> 另：他在第二小问中途还把目标地址念成「EFD4」，与他自己给出的「200C 减 0x38」的逐位过程矛盾——这一点按 **ASR 误转**处理（见上一段的 ⚠）。

4.5 第一题（C 代码 + 汇编，他说「是比较早的年份」）

```
题干要素：按字节编址；机器指令列从 00401020H 到 0040107FH（末行是 return，占 1 个字节）

(1) 概念题：这是精简指令集还是复杂指令集？→ 判据是【指令是否定长】
    定长 ⇒ 精简指令集（RISC）；不定长 ⇒ 复杂指令集（CISC）
    ⚠ 他只讲了判据，没有念出这道题的选项与结论，本资料不替他选边
(2) 机器代码占多少字节？
    0040107FH − 00401020H = 5FH ；再 +1 = 60H = 96 字节
    ★ 独立验算：0x7F − 0x20 = 0x5F，+1 = 0x60 = 96 ✓（他念的 96 也对：6 × 16 = 96）
    ⚠ 他专门提醒：能直接相减是因为末行 return 刚好只占 1 个字节；
      如果末行指令占 3 个字节，末地址还要在 7F 基础上再加 2
(3) 第 20 条指令：函数 f1(n)，n = 0（无符号 int）；CMP i, n−1 → 问 CF
    n − 1 = 0 − 1 = 0xFFFFFFFF（32 位全 1；可认为最高位有一个 1 供借位）
    i − (n−1) = 0 − 0xFFFFFFFF，是【小减大】⇒ CF = 1
    加法器路线：0 + ~0xFFFFFFFF + 1 = 0 + 0 + 1 = 1 ⇒ Cout = 0、Cn−1 = 0
                CF = SUB ⊕ Cout = 1 ⊕ 0 = 1 ；OF = Cn ⊕ Cn−1 = 0 ⊕ 0 = 0
    ★ 独立验算全部一致 ✓（他念的 CF = 1、OF = 0）
(4) 第 23 条（转写为「第二三条」）是 SHL（左移），能实现乘 2；问 f2（float 类型）能否用左移实现乘 2
    → 不能：浮点数有阶码，乘 2 只需把【阶码 +1】，整个左移会乱掉
```

「（第一问）这种就是一个概念题，这个就是**精简指令集和复杂指令集**，就是看这个**指令是否定长**」「不定长的话，它就是一个复杂指令集。」

「包括它问你这个**机器代码占多少个字节**，那你直接用最末尾的减去最开头的，**减完之后还要加一个 1**——有一个闭区间的话一定是（末减首加 1）」「减去 00401020，就算出来是一个 5F，5F 还要加一个 1，那就变成 16 进制的 60。」

「（第三问）你现在拿 i 减去（n−1），明显发现是一个**小的减去大的**，那小的减去大的一定是有借位的，**所以最后能算出来 CF 就是 1**。」「（但）计算机（内部）……本来是减法，SUB 传（1）进来……**把所有位都取反，再加一个 1**」「最后算出来（C_{out}）是 0，1 和 0 进行（异）或是 1。」

「（第五问）他说在 f2 中能否用 SHL 指令实现乘二，**你就说不能**，原因就是**浮点数它是有阶码的**，你乘二相当于（阶）码加一，**不能把整个进行左移**。」

4.6 第二题（同一个「阶乘」题干）：调用次数、跳转指令、偏移量与小端

```
题干要素：C 代码 f(n) = n > 1 ? n × f(n−1) : 1；按字节编址；
          第 16 行是 CALL f1；第 16 行指令地址 = 00401025H，该指令占 5 个字节；
          后 4 个字节为偏移量

(1) 算 f(10)：要依次算 f(9)、f(8) …… f(2)、f(1)
    → f 一共被调用【10 次】   ★ 独立验算：n = 10,9,…,1 共 10 次 ✓
(2) 哪条指令会递归调用自己 → 第 16 条（CALL f1）
(3) 条件转移只有一条 → 第 12 条（JLE，「小于等于就转移」）
(4) 一定会让程序跳转执行的三类指令 → CALL（第 16 条）、JMP（无条件转移）、RET（返回）
(5) 第 17 行的地址 = 00401025H + 5 = 0040102AH
    ★ 独立验算 0x25 + 5 = 0x2A ✓（一条 5 字节的指令写成 16 进制就是 10 个十六进制位，每 2 位 1 字节）
(6) 第 16 行采用相对寻址，求偏移量：
    这条指令执行完要跳回函数开头 00401000H
    0040102AH + 偏移量 = 00401000H ⇒ 偏移量 = −2AH = −42
    ★ 独立验算 0x0040102A − 42 = 0x00401000 ✓
    ⚠ 他中途先念成「−30 / −48」，随后自己纠正「算错了，这里应该是一个 −2A」
(7) 大端还是小端？偏移量 −42 的 32 位补码 = FFFFFFD6H
```

```
    内存里「先存了低字节 D6，再存 FF、FF、FF」→ 【小端方式】
    ★ 独立验算 −42 的补码 = 0x100000000 − 42 = 0xFFFFFFD6 ✓
(8) 为什么 f(13) 的结果 62 亿，返回值只有 19 亿？
    13! = 6227020800 ≈ 62 亿；32 位 int 最大 = 2^31 − 1 = 2147483647 ≈ 21 亿 → 溢出
    溢出后的值 = 13! − 2^32 = 1932053504 ≈ 19 亿（正好对上他说的 19 亿）
    改法：换成 long long（64 位整数）或 double
    ★ 独立验算：6227020800 − 4294967296 = 1932053504 ✓
(9) 有符号数乘法什么时候溢出？→ 乘积的高 33 位非全 0 或非全 1 就溢出（OF = 1）
    溢出后应该加一条【溢出陷入指令】
```

「第一问比较常规」「这个函数相当（于）执行 f10 就被调用 10 次」「它问你哪条（指令会递归调用），你可以回答这个第 16 条。」

「（条件）转移指令只有一条……这个 JLE 就是（less or equal），**小于等于会转移**」「那小于等于那就不能顺序执行，应该跳转执行。」「一定会跳转执行的就是 RET，还有这个 JMP，包括这个 CALL，**这三个是无条件转移**。」

「（第 17 行）刚好（每）两个 16 进制（位）刚好是一个字节，说明这一条指令占了 5 个字节，那它的下一条应该是加 5」「因为 5 加 5 是一个 10，16 进制的 A 表示 10，所以第 17 行的地址就是 0040102A。」

「最后它一定要重新偏移到函数这个开头去执行，这样你就知道这个偏移量（是 −2A）」「它是先存了低字节，先存了 D6，再存 FF、FF、FF，这样的——**它是一个小端方式**。」

「32 位 int 的表示范围是 −2 的 31 次方到 2 的 31 次方减一」「**2 的 31 次方你可以写成（2 的 10 次方）的 3 次方再乘 2**……这样算出来刚好是 20 亿」「**不相等的原因，那就是因为 int 超出了表示范围**」「可以换成 long long 类型，这个就 64 位的整数……但你也可以把它换成 double，都是可以的。」

5 真题年份认定与命题特点

结论：他本讲确实说出了几个年份，但没有一个是他课上精讲题目的年份

· 2017 「在真题里面 **17 年也是考的**，他问「适合按下标访问一维数组的是哪一个」，那就是**变址寻址**」——**这是他把年份和具体问法绑在一起的唯一一处。**

· 2018 「**18 年为什么专门考了变址寻址**……可能就那一年这一个题答得不太好，可能下年又出了一道这样的题，**相邻了两年**」——他说 17、18 两年相邻考变址。

· 2015 「**15 年这个题先不讲**，到时候会和 5.3 小节的一块讲，因为 **2015 年两道题目有关联**，单纯直接做这个题目是做不出来的，信息不全」——**本讲未展开。**

· 2019 「这个 **19 年这个题**和上面这个题是同一个题干……（考的是）哪些是块内地址、哪些是主存块号、问你是否命中，**都是 Cache 相关的**」——**他只说了题目归属，本讲未讲。**

· 2023 「**即便到了二三年，也是有考察过的**」——指汇编 / 控制流指令被考过，**没有具体题面。**另有 年份不详 「最近两年的一个，也是这种相关的题目」。

认定依据：以上年份**全部来自他的口述**（原话已逐句核对，见上引），本资料**没有独立核实题面**——他念的是口述版题干，不是原卷。因此**凡是他没点名年份的题目（第七题、第一题、第二题、课后 16 题 / 第九题等），一律标「年份不详」**，只保留他用的题号。要标年份必须回原卷核对，不能靠「看着像」。

他讲的命题特点（比年份更该记）：

① **同一题干拆成两道题：**「408 有时候它会把同一科目的两道题设置得有关联……单纯直接做这个题目是做不出来的，信息不全」（2015 年那两道、2019 年那道都用这个套路）。

② **还能跨科目关联：**「有时候会把不同科目这个（关联）……组成原理和操（作系）统结合一块，都是有

的。」

③ **今年抄往年**：「408 的真题它经常有时候会根据以往的一些大题重新出一道新的大题。」

④ **题干里的话都有用**：「**每一句话、题干的每一句话都是有用的**」「一定要注意：**题干上的每一个信息是怎么用上去的**」「选择题有时候会给干扰选项，大题的话绝大多数题目其实每个条件都是有用的。」

⑤ **新题会自带解释**：「像这种比较新的东西，他这个出题人就害怕你不太理解他的意思，后面还专门举了一个例子」「碰到题干比较长的话，**一定要耐下心**。」「（汇编那种）因为它是比较早的年份了，所以今后还是要记一下。」

课上精讲的题	他本讲的原话 / 编号	年份认定
变址取数组元素（真题选择题）	讲它时点名 **17 年 / 18 年**：「适合按下标访问一维数组的是哪一个，那就是变址寻址」	他说 17、18 年 但并未明说这道题就是那两年的那道，本资料不替它定年
第七题（175 页，条件转移 CZN）	「这是一个比较新的题」「他的出题方式和书上的知识点不一样」	年份不详 只说「比较新」
第一题（C 代码 + 汇编五列）	「这种考法可能不熟悉」「因为它是比较早的年份了」	年份不详 只说「比较早」
第二题（阶乘 f(10) / 小端） 课后 16 题 / 第九题	「第二题……它是用来算阶乘的」；末尾说 19 年那道与它同题干；课后题「课下也可以做一做」	年份不详 （「19 年那个题」指的是另一道与它同题干的 Cache 题，不是这道）

6 易错点清单

容易搞错的地方	为什么错	正确做法 / 老师原话
把「大写 B」当成字节	同一个符号在 408 里有两种意思	「有时候（大）B 表示字节，有时候又表示二进**位**」「就像 H 表示十六进制一样，这个大 B 表示二进（制）位」「这个不要把它当成（字）节」——要看上下文
按字编址时仍按字节数位宽	一个地址的大小变了	「**凡是碰到按字编址一定要小心一点**」「只要是按字编址，一定得告诉你每个字多大」
把机器字长 / 指令字长 / 存储字长混用	题目只说「字长」	「**到底它（指）是哪个意思，你可能需要具体到题目去分析**」「缺哪个你就当中（用）哪个」
相对寻址直接用「PC + A」	真题口径不同	「**先用 PC 加（指令长度），PC 要先取到下一条指令的地址，然后再去加偏移量**」；还要看「偏移量到底有没有系数」
把无符号地址和有符号偏移量直接相加	一个无符号、一个有符号	「**你把一个无符号数和有符号数是不能直接参与运算的**」「一定要把它的真值先求出来」
变址取数组时忘了「乘元素大小」	变址寄存器里放的是下标	「有时候这个变址寄存器**可能还要乘上一个东西**，乘上一个东西再加上这个指令里面的形式地址」

容易搞错的地方	为什么错	正确做法 / 老师原话
把「检测位」当成标志位本身去填	CZN 是「检测哪几位」的开关	「**千万不要去回答 CF 的数值和 ZF 的数值，人家问的是检测位**」「C 是 1 去检测 CF，Z 是 1 检测 ZF」
以为 CF 只看结果正负	CF 是进位 / 借位	「CF 既可以表示进位，也可以表示借位：加法表示进位，减法表示借位」；无符号比较用 CF（+ZF）
区间里有多少个数忘了 +1	闭区间	「问你一个区间 A 到 B 有多少个数，**一定是减完之后要加 1 的**」；偏移量向后共 127 条
算机器代码字节数时直接末 − 首	漏了闭区间的 +1	「减完之后还要加一个 1」；还要确认末行指令本身占几个字节（占 3 字节就要再 +2）
「哪些」当成「哪个」 / 写一大堆过程	习惯用词；没有过程分	「「哪些」是多个、「哪个」一般就一个……最后还是按你的分析过程为主」；「**基本没有过程分**，大多数只写答案就行」

7 考场速查卡

【一】三个字长（问什么答什么）
机器字长 = 一次运算处理的位数 = 通用寄存器位数 = ALU 位数
指令字长 = 一条指令的长度　　存储字长 = 一个存储单元位数 = MDR 位数 = 数据线位数
MAR 位数 = 地址线位数 = $\log_2$(地址总个数)　　MDR 位数 = 数据线位数 = 存储字长

【二】编址方式（按字节 / 按字）
按字节：1 个地址 = 1 字节，一个 32 位存储单元占 4 个地址
按字　：1 个地址 = 1 个字（= 存储字长），必须知道「一个字多大」
判法：PC 加几 = 一条指令占几个地址；再拿「一条指令几个字节」去比 → 定编址方式
⚠「80% 的题目都是按字节」，碰到按字编址先找陷阱

【三】十种寻址方式与 EA
隐含：操作数在 ACC（一地址指令也能有两个操作数）
立即：数在指令里（地址码放的不是地址而是数据）
直接：EA = A　　间接：EA = (A)　　X = ((A))
寄存器：EA = R（数在 R）　寄存器间接：EA = (R)　X = ((R))
相对：书上 PC + A　　基址：BR + A（BR 不变、地址码变）→ 程序浮动
变址：IX + A（IX 变、A 不变）→ 数组　　堆栈：栈顶隐含，了解一下即可
⚠ 真题里相对寻址 = PC + 指令长度 + 系数 × 偏移量；变址取数组要【乘元素大小】

【四】相对转移的三件事
① 目标 = (PC) + 指令长度 + 系数 × 偏移量
② 偏移量是 8 位补码 → −128 ~ +127；−1 原地踏步；0 及以上向前
③ 向后可用 −128 ~ −2 → 共 127 条（区间计数一律「末 − 首 + 1」）
⚠ 有符号偏移量先求真值，再化成无符号量去做加减（200C − 0x38 = 1FD4）

【五】标志位与加法器
CF 进位 / 借位（无符号）　OF 溢出（有符号）　SF 符号位　ZF 零标志
CF = SUB ⊕ Cout　　OF = Cn ⊕ Cn−1　　（减法：操作数取反 + Cin = 1）
无符号比较：a < b ⇔ CF = 1 ；a = b ⇔ CF = 0 且 ZF = 1 ；a > b ⇔ CF = 0 且 ZF = 0
有符号乘法溢出：乘积高 33 位非全 0 或非全 1 ；溢出后加【溢出陷入指令】

【六】汇编与机器级表示（记缩写 ↔ 全称）
传送 MOV / PUSH（入栈，参数传递）/ POP（出栈）—「栈增长方向与内存地址增长方向相反」
运算 ADD SUB INC DEC IMUL　逻辑 AND OR NOT XOR SHL SHR
控制流 JMP（无条件）Jcc（JE JZ JNE JG JGE JL JLE）CMP（先做减法）CALL（调用）RET（返回）
机器码拼法：操作码 | 寻址方式 | 寄存器号 …… 每 4 位一组转 16 进制
执行结果写回【目的操作数】；别忘了寄存器自增这类隐含动作

如果只记五件事

① **书上三套 EA 公式在真题里都要改口径**：相对 = PC + 指令长度 + 系数 × 偏移量；变址取数组 = 首地址 + 下标 × 元素大小；
② **判编址方式看「PC 加几」**，按字编址必有陷阱；三个「字长」缺哪个用哪个；
③ **偏移量是有符号的**：−128 ~ +127，向后只有 −128 ~ −2（127 条），且要先求真值再和无符号地址相加；
④ **CF/OF 两条路都要会**：观察法（小减大 → CF = 1）用于选择，公式法（SUB $\oplus C_{out}$ / $C_n \oplus C_{n-1}$）用于大题写过程；
⑤ **大题「每一句话都有用」**，而且往往 1 小问 1–2 分、没有过程分——答案写清楚比过程写详细更值钱。

附：本资料的来源与边界

来源	内容	可靠性与边界
B 站《408 各科大题知识点强化课》课程第 7 讲「计算机组成原理 · 指令寻址与汇编」，分 P9，音频 **3 小时 26 分 45 秒**	§1 – §7 的全部	本地 Whisper 转写，讲稿 `.md` 2509 行 / 约 13.4 万字节，覆盖 `[00:00:19]`–`[03:26:32]`，即**全程**；其中 `[01:31:04]`–`[01:40:15]` 是课间休息（约 9 分钟），另有五段安静读题时段（`[00:47:55]`、`[00:59:38]`、`[01:42:50]`、`[02:36:23]`、`[03:01:02]` 起，合计约 14 分钟无口述）。**本讲全程无板书**，位串、除法拆位、补码转换全凭口述，按他念出的运算顺序还原，并把每一处算式独立重算过一遍（见下）。
教材 / 通用常识（本资料的补充）	「扩展操作码」「指令周期 / 机器周期 / 取指周期 / 间址周期 / 执行周期」「微操作 / 微指令」「流水线的具体内容」「MESI 之类一致性」等名称，仅出现在「本讲未讲」的说明里，**未展开任何内容**	**教材补充，非老师口径。**这些术语在全讲转写稿里出现 **0 次**（「扩展」6 次全是「符号扩展」，「中断」1 次是零地址指令的举例）。本资料**没有替他把内容补出来。**

★ 第五步自查（一）：算式独立验算清单 —— 老师念出的每一个算式都重算过

算式（出处）	讲稿里的值	独立验算
数组题：8i = 100H → i；元素地址	i = 32；元素地址两处都转写成「2000」	✓ **256 ÷ 8 = 32**；✗ **元素地址应为 2100H**——若真是 2000H 则 i = 0，讲稿自相矛盾。只保留「首地址 2000H」与「i = 32」
机器码拼接 0010|001|100|010|101	2315	✓ **0x2315**，与他念的一致
5678H + 1234H / R5 自增	68AC / 5679	✓ **0x68AC、0x5679**
200C + 2（CZN = 011 不转移）	200E	✓ **0x200E**
偏移量 E3 的真值与乘 2	−29（他说「−58」是乘 2 之后）	✓ 取反 1C + 1 = 1D = 29 → −29；2 × (−29) = −58
目标 = 200C + 2 + 2 × (−29)	转写为「EFD4」	✗ **应为 1FD4H**：0x200E − 58 = 0x1FD4；逐位减法 C−8 = 4、0−3 借位得 D、被借的 0 变 F、2 被借变 1 → 1FD4（ASR 误转）
8 位补码表示范围	「−128 到 −127」	✗ **口误**：应为 **−128 ~ +127**（他同句给出的公式「2^7 − 1」= 127 是对的）→ 已加框声明并按正确值呈现
向后跳转条数（−128 ~ −2）/「3 到 6 有几个数」	127 条 / 4 个	✓ (−2) − (−128) + 1 = 127；6 − 3 + 1 = 4
机器代码字节数：7F − 20 + 1	5F + 1 = 60H = 96	✓ **0x5F + 1 = 0x60 = 96**（6 × 16 = 96 也对）
CF / OF（第一题 i − (n−1)）	CF = 1、OF = 0	✓ 小减大 → 借位；加法器路线 0 + 0 + 1 = 1 → C_{out} = 0、C_{n-1} = 0 → CF = 1⊕0 = 1、OF = 0⊕0 = 0
f(10) 的调用次数	10 次	✓ n = 10, 9, …, 1 共 10 次
第 17 行地址：00401025 + 5	0040102A	✓ **0x25 + 5 = 0x2A**
偏移量：0040102A → 00401000	先念 −30（−48），后自己改成 −2A	✓ **只有 −2AH（−42）自洽**：0x0040102A − 42 = 0x00401000
−42 的 32 位补码 / 13! 与 int 溢出	FFFF FFD6 / 62 亿；返回值 19 亿；int 上限约 20 亿	✓ 2^{32} − 42 = 0xFFFFFFD6；13! mod 2^{32} = 1932053504 ≈ 19 亿；2^{31} − 1 ≈ 21 亿（他按 $2^{31} \approx 2 \times 10^9$ 粗算成 20 亿）
小例子：相对寻址 PC 100 + 2 / 程序浮动 100 + 99、1000 + 99 / 128KB ÷ 2B	102 / 199、1099 / 64K→MAR 16 位 / 1 兆 / 16 条、8 个	✓ 全部一致：102；199、1099；65536 = 2^{16}；1048576 B = 1MB；2^4 = 16、2^3 = 8（反向 16 个寄存器 → 4 位）

算式（出处）	讲稿里的值	独立验算
/ 2^{20} 地址 × 1B / 4 位操作码、3 位寄存器号		

结论：讲稿里真正**算错**的只有一处（8 位补码范围念成「−128 到 −127」）——已在 §4.4 加框声明为**老师口误**并按正确值呈现；另有**两处转写硬伤**（`EFD4` 应为 `1FD4`、元素地址 `2000` 应为 `2100H`，后者导致原句自相矛盾）——已按可自洽的值呈现并标注。**其余全部与老师念出的一致，未发现新的口误。**

★ 第五步自查（二）：引语回对 —— 转写含糊、按上下文推断或存疑处（逐条列出）

① `[00:55:59]` / `[00:56:11]`「取了元素的地址是 2000 億」→「億」应是他念的「H」，但**两处都转成 2000**，与同段「8i 等于一百」矛盾，按他的方程应为 **2100H**；本资料只保留「首地址 2000H」「i = 32」（§4.1）。

② `[01:13:19]`「**MOS** 叫寻址方式」字段名转写不明（疑为 MOD）；`[01:14:15]`「那总共 6 位 0 到 5」与 `[01:12:49]`「操作码是 4 位，12,13,14,15」不可辨读 → 只采用「寻址方式字段 3 位 → 8 个寄存器」「4 位操作码 → 16 条指令」（§4.3）。

③④ `[02:06:17]`「**EF 第四**」应为 **1FD4H**（「1F」被转成「EF」）；`[01:57:11]`「8 位补码的表示范围……−128 到 −127」上限应为 **+127**（口误）。两处均已在上表中独立验算，并在 §4.4 加框声明。

⑤ `[03:12:29]`「A 是一个 16 进制的 **−30**」「真值就是一个 **−48**」→ 他随后自己纠正「这里算错了，应该是一个 −2A」，本资料按 **−2AH（−42）**（§4.6）。

⑥ `[01:22:17]`「相当于现在有 **12345**」、「`[01:23:21]` 内存也是有 **1231234** 里面放的是 5678」→ 前者应为 R4 = **1234H**、后者应为「内存的 1234H 单元」（按他自己后文的 5678H + 1234H = 68ACH 反推确认）；`[01:30:16]`「哪些寄存器会变的就是 **101 号**」转写含糊，按其分析只有 **R5** 会变（§4.2）。

⑦ `[02:26:51]` 段末连续 7 次重复「有时候会把数据减少」→ 整段崩坏，只采用可辨读的「同一科目两道题关联 / 跨科目关联」两句；`[01:49:12]`「选择贴一般就 6 到 50 分」数字崩坏，只保留「60–70 分钟」这一条（§1、§5）。

⑧ `[00:01:22]`「考评最高的是开始第三章的开始和第四章」→ 整句混乱、未采用，只采用同段可辨读的「第四章也是一个大题的高频考点」（§1）。

⑨ `[01:10:33]` 附近「按字编址、每个字 32 位」的改题他只说「你就应该这样去做」、**没念出结果**，§4.3 的 4MB 是本资料按其口径补算的；第七题的「E3」「CZN = 011」「检测位 110」三处转写清晰且与算式互印，无存疑。

⑩ 他把「变址」念成「变质 / 变子」、「基址」念成「机子 / 机质」，已统一校正（都有 IX / BR 的算式跟着，不会混淆）。

老师没有讲 / 本讲没有覆盖的内容（对照你给的大纲）

① **扩展操作码——本讲未讲（全讲 0 次）。**「扩展」二字只出现在「**符号扩展**」（offset 4 位 → 8 位）里；他没有讲操作码扩展、扩展操作码的位数分配，也没有讲「不定长操作码」的设计方法。他讲到操作码时只有两句：「指令由操作码和地址码构成」「问能表示多少条指令，只看操作码位数」。

② **指令周期与机器周期——本讲未讲（0 次）。**他只在第一章回顾里讲了**时钟周期**（「计算机最小的执行时间单位」）与 CPI；「取指周期 / 间址周期 / 执行周期 / 中断周期 / 机器周期」全部没有出现，讲执行流程时用的是「一条指令会先从 PC 把地址取到 MAR……」，**没有给这些阶段命名。**

③ 微操作 / 微指令 / 微程序——本讲未讲（0 次）。他说「下次上课就讲第五章 CPU 那里」。**④ 流水线只被提及、没有讲：**「后面像流水线有的题目本质上也是考指令」——仅此一句，无内容。**⑤ 中断只被提及一次：**作为「零地址指令」的例子（「关机、中断，然后空操作，都不需要地址码」），没有讲中断机制。**⑥ 本讲没有再讲计组的分值分布：**关于「考什么」只有 §1 引的那几句。**⑦ 三类题被明确留到课下 / 回放：**2015 年那两道关联题（他说与 5.3 小节一起讲）、2019 年那道与阶乘同题干的 Cache 题、以及「最近两年」那一道——**本讲只报题、未讲。⑧ 无授课内容的时段：**课间 `[01:31:04]`–`[01:40:15]`，以及本附录开头列出的五段读题时段。

本资料与你的大纲的差异（一句话版）

你列了、老师没讲的：扩展操作码、指令周期与机器周期（均为 0 次，已在上框标注「本讲未讲」）。**你列了、老师讲得更多的：**寻址方式（十种，多出隐含寻址与堆栈寻址）、指令格式（多出 0～4 地址指令的逐类讲法）、汇编（多出 185 页整张指令表 + 四道大题）。**你没列、老师讲了不少的：**第一章回顾整块（MAR/MDR、三总线、编址方式、三个字长、时钟周期与 CPU 执行时间、PC/IR/ACC/标志寄存器）、寻址特征字段、CISC/RISC（定长判据）、CF/OF/SF/ZF 与加法器求 CF/OF、补码与原码互转、float 格式与「浮点乘 2 只加阶码」、int 溢出与 long long/double、有符号乘法溢出判据与溢出陷入指令、小端方式、符号扩展 / 移位器 / 多路选择器那张数据通路图，以及答题与时间策略。

整合自 3 小时 26 分 45 秒网课讲稿（本地 Whisper 转写，全程无板书；位串、除法拆位、补码转换均按口述还原）· 未上传任何材料到第三方服务 · 全部例题、数值与算式均回对过讲稿原文并独立重算，发现的 1 处老师口误与 2 处转写硬伤已在 §4.4 与附录中逐一标注